

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage:40%

Dr

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze

7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

Code of Conduct:

I AAMEENA I(192511284) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

		satisfying all constraints			
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application ; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWERS:

QUESTION 1 – QUERY EXECUTION PLAN AND HEURISTIC OPTIMIZATION

Aim

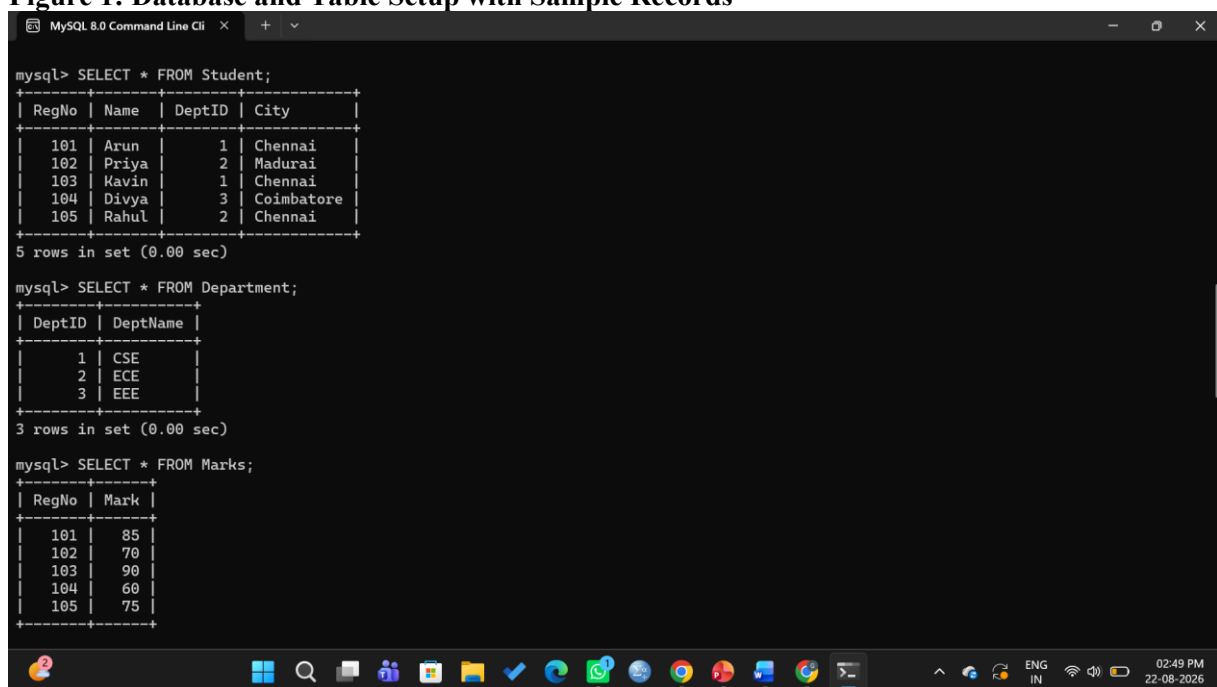
To analyze a complex SQL query containing joins and a subquery, obtain its execution plan, and optimize it using heuristic optimization techniques.

1. Database and Table Setup

A separate database was created with three relations: Student, Department, and Marks.

Sample records were inserted into the tables to demonstrate query execution and optimization.

Figure 1: Database and Table Setup with Sample Records



The screenshot shows the MySQL Command Line Client interface with three tables displayed:

Student Table:

RegNo	Name	DeptID	City
101	Arun	1	Chennai
102	Priya	2	Madurai
103	Kavin	1	Chennai
104	Divya	3	Coimbatore
105	Rahul	2	Chennai

5 rows in set (0.00 sec)

Department Table:

DeptID	DeptName
1	CSE
2	ECE
3	EEE

3 rows in set (0.00 sec)

Marks Table:

RegNo	Mark
101	85
102	70
103	90
104	60
105	75

2. Complex SQL Query

The query retrieves students who satisfy the following conditions:

- The student is from Chennai.
- The student belongs to the CSE department.
- The student's mark is greater than the average of marks above 80.

The query uses two joins and a nested subquery to obtain the required result.

For the given data, the average of marks greater than 80 is:

$$\text{Average} = (85 + 90) / 2 = 87.5$$

Therefore, the student with a mark greater than 87.5 is Kavin, with a mark of 90.

Figure 2: Execution of the Complex SQL Query and Result

```
MySQL 8.0 Command Line CL  x  +  -
5 rows in set (0.00 sec)

mysql> SELECT S.Name, D.DeptName, M.Mark
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> JOIN Marks M
-> ON S.RegNo = M.RegNo
-> WHERE S.City = 'Chennai'
-> AND D.DeptName = 'CSE'
-> AND M.Mark > 6
-> SELECT AVG(M2.Mark)
-> FROM Marks M2
-> WHERE M2.Mark > 80
-> );
+-----+-----+-----+
| Name | DeptName | Mark |
+-----+-----+-----+
| 1    | CSE      | 90   |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> EXPLAIN
-> SELECT S.Name, D.DeptName, M.Mark
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> JOIN Marks M
-> ON S.RegNo = M.RegNo
-> WHERE S.City = 'Chennai'
-> AND D.DeptName = 'CSE'
-> AND M.Mark > 6
-> SELECT AVG(M2.Mark)
-> FROM Marks M2
-> WHERE M2.Mark > 80
-> );
```

3. Query Execution Plan

A query execution plan represents the sequence of operations performed by the DBMS to execute a query. The EXPLAIN command was used to analyze how the database system processes the given query.

The execution involves:

1. Reading the required relations.
2. Applying selection conditions.
3. Evaluating the subquery.
4. Performing join operations.
5. Producing the required output attributes.

Figure 3: Query Execution Plan Obtained Using EXPLAIN

```
mysql> EXPLAIN
-> SELECT S.Name, D.DeptName, M.Mark
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> JOIN Marks M
-> ON S.RegNo = M.RegNo
-> WHERE S.City = 'Chennai'
-> AND D.DeptName = 'CSE'
-> AND M.Mark > 6
-> SELECT AVG(M2.Mark)
-> FROM Marks M2
-> WHERE M2.Mark > 80
-> );
+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+
| 1 | PRIMARY | S | NULL | ALL | PRIMARY | PRIMARY | 4 | dbms_q1.S.DeptID | 5 | 20.00 | Using where |
| 1 | PRIMARY | D | NULL | eq_ref | PRIMARY | PRIMARY | 4 | dbms_q1.S.DeptID | 1 | 33.33 | Using where |
| 1 | PRIMARY | M | NULL | ALL | NULL | NULL | NULL | NULL | 5 | 20.00 | Using where; Using join buffer (hash join) |
| 2 | SUBQUERY | M2 | NULL | ALL | NULL | NULL | NULL | NULL | 5 | 33.33 | Using where |
+-----+-----+-----+
4 rows in set, 1 warning (0.00 sec)

mysql>
```

4. Initial Query Execution Plan

The initial execution plan can be represented as:

```
SELECT
  |
  Projection
Name, DeptName, Mark
  |
  JOIN
 /  \
JOIN  Marks
 /  \  |
 /  \ Selection
 /    \ Mark > 80
Student Department
 |      |
Selection Selection
City='Chennai' DeptName='CSE'
```

The initial plan processes the relations and applies the required conditions before producing the final result.

5. Heuristic Optimization

Heuristic optimization applies predefined rules to improve query performance by reducing unnecessary processing and intermediate results.

5.1 Selection Pushdown

Selection conditions are moved closer to the base relations.

σ City='Chennai' (Student)

σ DeptName='CSE' (Department)

σ Mark > 80 (Marks)

Benefit: Unwanted tuples are eliminated early, reducing the amount of data involved in subsequent operations.

5.2 Projection Pushdown

Only the attributes required for selection, joining, and final output are retained.

Student → RegNo, Name, DeptID

Department → DeptID, DeptName

Marks → RegNo, Mark

Benefit: Unnecessary attributes are removed early, reducing memory usage and data processing.

5.3 Join Ordering

The filtered and smaller relations are joined first to reduce the size of intermediate results.

Filtered Student

+

Filtered Department

↓

First Join

↓

Filtered Marks

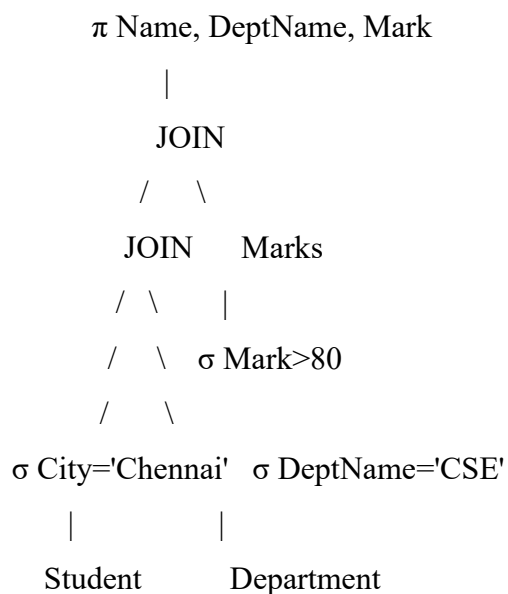
↓

Final Join

Benefit: Smaller intermediate relations reduce processing time, memory requirements, and disk I/O.

6. Optimized Query Execution Plan

The optimized plan is:



The optimized plan performs selections and projections before the join operations, thereby reducing the size of intermediate relations.

7. Comparison

Before Optimization	After Optimization
More tuples are processed	Fewer tuples are processed
Unnecessary attributes may be carried	Only required attributes are retained
Larger intermediate results	Smaller intermediate results
Higher processing cost	Lower processing cost
More disk I/O	Reduced disk I/O

8. Result

The SQL query was successfully analyzed and its execution plan was obtained using EXPLAIN. Heuristic optimization was applied using selection pushdown, projection pushdown, and join ordering.

Conclusion

Heuristic optimization reduces the number of tuples, attributes, and intermediate results processed by the DBMS. Therefore, the optimized query requires less processing, memory, and disk I/O while producing the same required result.

QUESTION 2 – COST-BASED OPTIMIZATION OF JOIN OPERATIONS

Aim

To analyze different join algorithms and select the most efficient join technique using cost-based query optimization.

1. Given Information

Consider two relations **R** and **S** with the following properties:

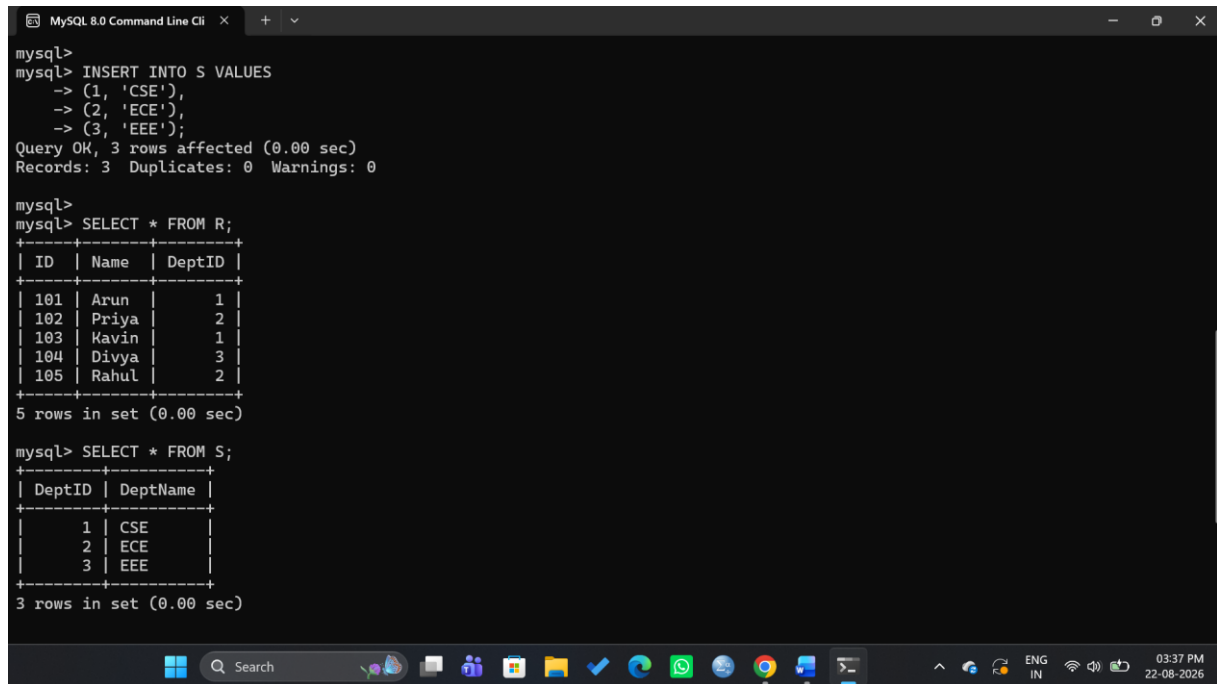
Parameter	Relation R	Relation S
Number of tuples	10,000	5,000
Number of pages	100	50

Available memory buffers = 10 pages

The join algorithms considered are:

1. Nested-Loop Join
2. Sort-Merge Join
3. Hash Join

Figure 1: Creation and Population of Relations R and S



```
mysql>
mysql> INSERT INTO S VALUES
  -> (1, 'CSE'),
  -> (2, 'ECE'),
  -> (3, 'EEE');
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> SELECT * FROM R;
+----+-----+-----+
| ID | Name  | DeptID |
+----+-----+-----+
| 101 | Arun  | 1      |
| 102 | Priya | 2      |
| 103 | Kavin | 1      |
| 104 | Divya | 3      |
| 105 | Rahul | 2      |
+----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT * FROM S;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| 1      | CSE      |
| 2      | ECE      |
| 3      | EEE      |
+-----+-----+
3 rows in set (0.00 sec)
```

2. Nested-Loop Join

For Block Nested-Loop Join, the cost is:

$$\text{Cost} = B(R) + \lceil B(R)/(M - 2) \rceil \times B(S)$$

Where:

- $B(R) = 100$ pages
- $B(S) = 50$ pages
- $M = 10$ buffers

Therefore,

$$\text{Cost} = 100 + \lceil 100/8 \rceil \times 50$$

$$\text{Cost} = 100 + 13 \times 50$$

$$\text{Cost} = 750 \text{ page I/Os}$$

Thus, the estimated cost of Nested-Loop Join is 750 page I/Os.

3. Sort-Merge Join

Sort-Merge Join consists of two main phases:

1. **Sorting:** Both relations are sorted based on the join attribute.
2. **Merging:** The sorted relations are scanned and matching tuples are combined.

The total cost depends on the number of pages, available memory buffers, and the number of sorting passes.

For the given relations, Sort-Merge Join requires additional I/O for sorting and merging.

Therefore, its cost is compared with the other join methods before selecting the final

execution plan.

4. Hash Join

For a Grace Hash Join, the approximate cost is:

$$\text{Cost} = 3 \times (B(R) + B(S))$$

Substituting the values:

$$\text{Cost} = 3 \times (100 + 50)$$

$$\text{Cost} = 3 \times 150$$

$$\text{Cost} = 450 \text{ page I/Os}$$

Thus, the estimated cost of Hash Join is 450 page I/Os.

5. Cost Comparison

Join Algorithm	Estimated Cost
Nested-Loop Join	750 page I/Os
Sort-Merge Join	Depends on sorting passes
Hash Join	450 page I/Os

For the given equality join, Hash Join provides the lowest estimated cost among the calculated alternatives.

6. SQL Demonstration

The join operation was implemented using the following SQL query:

```
SELECT R.ID, R.Name, S.DeptName
```

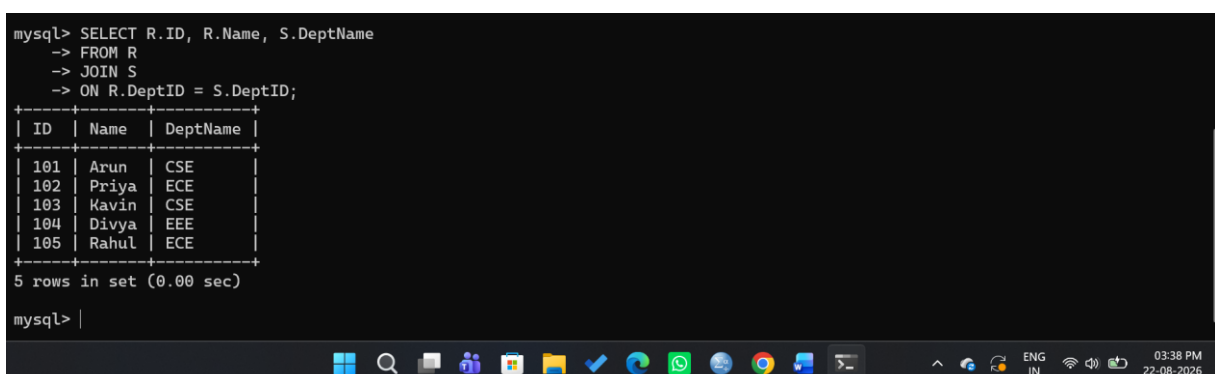
```
FROM R
```

```
JOIN S
```

```
ON R.DeptID = S.DeptID;
```

The query joins relations **R** and **S** using the common attribute DeptID.

Figure 2: Execution of Join Query and Result



```
mysql> SELECT R.ID, R.Name, S.DeptName
-> FROM R
-> JOIN S
-> ON R.DeptID = S.DeptID;
+----+-----+-----+
| ID | Name  | DeptName |
+----+-----+-----+
| 101 | Arun  | CSE      |
| 102 | Priya | ECE      |
| 103 | Kavin | CSE      |
| 104 | Divya | EEE      |
| 105 | Rahul | ECE      |
+----+-----+-----+
5 rows in set (0.00 sec)

mysql>
```

7. Cost-Based Decision

Cost-based optimization compares different execution strategies using factors such as:

- Number of tuples
- Number of pages
- Disk I/O cost
- Available memory
- Sorting cost
- Join processing cost

For the given relations:

Nested-Loop Join = 750 page I/Os

Hash Join = 450 page I/Os

Since Hash Join has the lower estimated cost, it is selected as the preferred join algorithm for this equality condition.

8. Advantages of Hash Join

- Efficient for equality-based joins.
- Requires fewer I/O operations in suitable cases.
- Avoids repeated scanning of the inner relation.
- Provides efficient processing for large relations.

9. Result

The costs of Nested-Loop Join, Sort-Merge Join, and Hash Join were analyzed. Based on the estimated I/O cost, Hash Join is selected as the most efficient option for the given equality join.

10. Conclusion

Cost-based optimization helps the DBMS choose an efficient execution plan by comparing the estimated costs of different join algorithms. For the given relations, Hash Join has a lower estimated cost than Block Nested-Loop Join and is therefore preferred.

QUESTION 3 – JDBC CONNECTIVITY WITH MYSQL

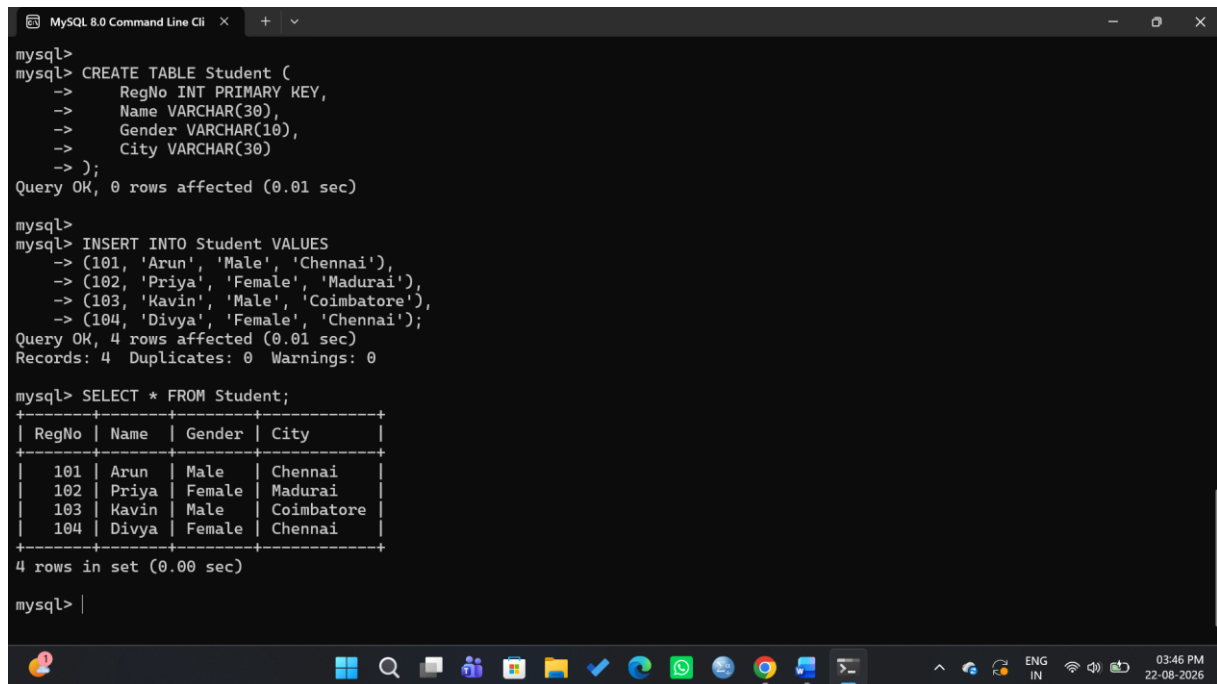
Aim

To establish JDBC connectivity between a Java application and a MySQL database and execute a parameterized SQL query using Prepared Statement.

1. Database and Table Creation

A MySQL database named DBMS_Q3 was created with a Student table containing student registration number, name, gender, and city. Sample records were inserted into the table.

Figure 1: Student Table with Sample Records in MySQL



```
mysql> CREATE TABLE Student (  
->   RegNo INT PRIMARY KEY,  
->   Name VARCHAR(30),  
->   Gender VARCHAR(10),  
->   City VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> INSERT INTO Student VALUES  
-> (101, 'Arun', 'Male', 'Chennai'),  
-> (102, 'Priya', 'Female', 'Madurai'),  
-> (103, 'Kavin', 'Male', 'Coimbatore'),  
-> (104, 'Divya', 'Female', 'Chennai');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM Student;  
+-----+-----+-----+-----+  
| RegNo | Name  | Gender | City    |  
+-----+-----+-----+-----+  
| 101   | Arun  | Male   | Chennai |  
| 102   | Priya | Female | Madurai |  
| 103   | Kavin | Male   | Coimbatore |  
| 104   | Divya | Female | Chennai |  
+-----+-----+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql> |
```

2. JDBC Connectivity

JDBC (**Java Database Connectivity**) is an API that allows Java applications to communicate with relational databases such as MySQL.

The JDBC program performs the following operations:

1. Loads the MySQL JDBC driver.
2. Establishes a connection with the MySQL database.
3. Creates a PreparedStatement.
4. Assigns a value to the parameter using setString().
5. Executes the SQL query.
6. Retrieves the result using ResultSet.
7. Closes the database resources.

3. JDBC Program

```
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/DBMS_Q3";
        String username = "root";
        String password = "YOUR_PASSWORD";

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                url, username, password
            );

            System.out.println("Database Connected Successfully!");

            String sql = "SELECT * FROM Student WHERE City = ?";
            PreparedStatement ps = con.prepareStatement(sql);

            ps.setString(1, "Chennai");

            ResultSet rs = ps.executeQuery();

            System.out.println("\nStudents from Chennai:");
            while (rs.next()) {
                System.out.println(
                    rs.getInt("RegNo") + " " +
                    rs.getString("Name") + " " +
                    rs.getString("Gender") + " " +
                    rs.getString("City")
                );
            }
        }
    }
}
```

```

rs.close();
ps.close();
con.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
}

```

Figure 2: JDBC Connectivity and Parameterized Query Execution

```

Database Connected Successfully!

Students from Chennai:
101 Arun Male Chennai
104 Divya Female Chennai

```

4. Role of PreparedStatement

PreparedStatement is used to execute parameterized SQL queries. In the given program, the ? symbol acts as a parameter placeholder.

SELECT * FROM Student WHERE City = ?

The value is supplied using:

ps.setString(1, "Chennai");

This approach improves security by reducing the risk of **SQL injection** and allows the same SQL statement to be executed with different parameter values.

5. Important JDBC Components

Component	Function
DriverManager	Establishes a connection with the database
Connection	Represents the database connection
PreparedStatement	Executes parameterized SQL queries
ResultSet	Stores the retrieved records
executeQuery()	Executes a SELECT query
setString()	Assigns a value to a parameter

6. Result

The Java application successfully established a connection with the MySQL database and retrieved the required student records using a parameterized query.

Conclusion

JDBC provides a standard mechanism for connecting Java applications with MySQL databases. Prepared Statement enables secure and efficient execution of parameterized SQL queries and helps prevent SQL injection attacks.

QUESTION 4 – CONFLICT SERIALIZABILITY USING PRECEDENCE GRAPH

Aim

To determine whether a given schedule of concurrent transactions is conflict-serializable using a precedence graph.

1. Given Schedule

The following schedule consists of four concurrent transactions **T1, T2, T3, and T4** operating on data items **X and Y**.

Step	Transaction Operation
1	T1: R(X)
2	T1: W(X)
3	T2: R(X)
4	T2: W(X)
5	T3: R(Y)
6	T3: W(Y)
7	T4: R(Y)
8	T4: W(Y)
9	T2: R(Y)
10	T2: W(Y)
11	T1: R(Y)
12	T1: W(Y)

2. Identification of Conflicting Operations

Two operations are said to be conflicting when:

1. They belong to different transactions.
2. They access the same data item.
3. At least one of them is a write operation.

The conflicting operations produce the following precedence relationships:

- **T1 → T2** because T1 writes X before T2 accesses X.

- **T3 → T4** because T3 writes Y before T4 accesses Y.
- **T4 → T2** because T4 writes Y before T2 accesses Y.
- **T2 → T1** because T2 writes Y before T1 accesses Y.

3. Precedence Graph

The precedence graph contains four vertices:

T1, T2, T3, T4

The directed edges are:

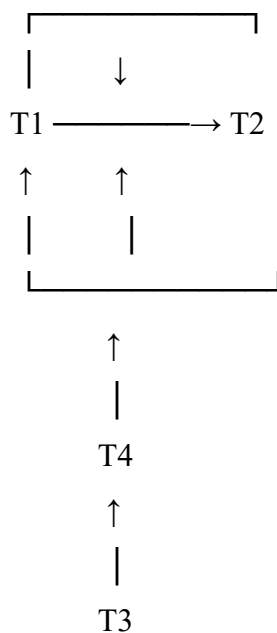
T1 → T2

T3 → T4

T4 → T2

T2 → T1

Precedence Graph for the Given Transaction Schedule:



4. Cycle Detection

From the precedence graph, the following cycle is present:

T1 → T2 → T1

Therefore, the graph is cyclic.

5. Serializability Analysis

A schedule is conflict-serializable if and only if its precedence graph is acyclic.

In this case, the precedence graph contains a cycle:

T1 → T2 → T1

Hence, the given schedule is not conflict-serializable.

6. Result

The precedence graph for the given schedule was constructed and analyzed. Since the graph contains a cycle between T1 and T2, the given schedule is not conflict-serializable.

7. Conclusion

Conflict serializability can be determined by constructing a precedence graph and checking for cycles. An acyclic graph represents a conflict-serializable schedule, while a cyclic graph represents a non-conflict-serializable schedule. Therefore, the given schedule is not conflict-serializable.

QUESTION 5 – BASIC TWO-PHASE LOCKING (2PL)

Aim

To demonstrate the application of the Basic Two-Phase Locking (2PL) protocol on a given set of transactions and identify and resolve deadlocks.

1. Given Transactions

Consider two concurrent transactions T1 and T2 operating on data items A and B.

Transaction	Operations
T1	Read(A), Write(A), Read(B), Write(B)
T2	Read(B), Write(B), Read(A), Write(A)

2. Two-Phase Locking Protocol

Basic 2PL divides a transaction into two phases:

Growing Phase

- The transaction can acquire locks.
- It cannot release any lock.

Shrinking Phase

- The transaction can release locks.
- It cannot acquire any new lock.

Therefore:

Growing Phase → Shrinking Phase

3. Locking Schedule

The transactions may execute as follows:

T1: LOCK-X(A)

READ(A)

WRITE(A)

LOCK-X(B)

READ(B)

WRITE(B)

UNLOCK(B)

UNLOCK(A)

COMMIT

T2: LOCK-X(B)

READ(B)

WRITE(B)

LOCK-X(A)

READ(A)

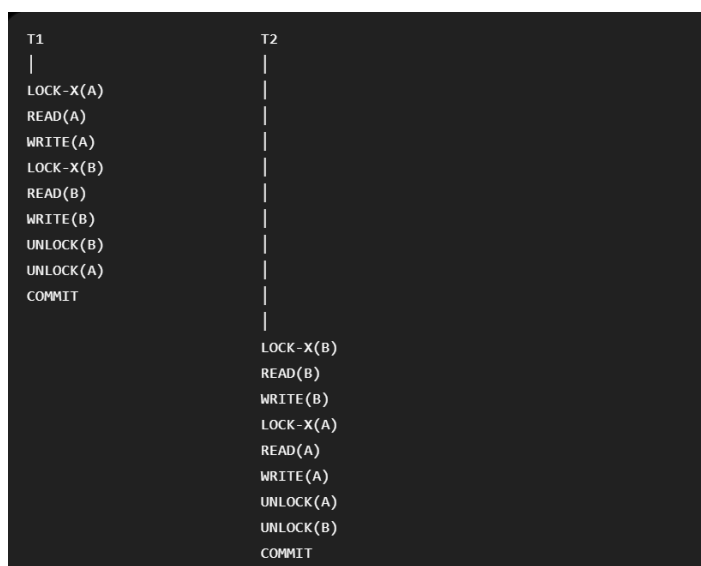
WRITE(A)

UNLOCK(A)

UNLOCK(B)

COMMIT

Figure 1: Application of Basic Two-Phase Locking to Transactions T1 and T2



4. Deadlock Condition

A deadlock can occur when both transactions hold a lock and wait for a lock held by the other transaction.

For example:

T1 holds X(A) → requests X(B)

T2 holds X(B) → requests X(A)

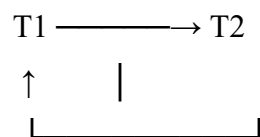
This creates a circular wait:

T1 → T2 → T1

Therefore, a deadlock occurs.

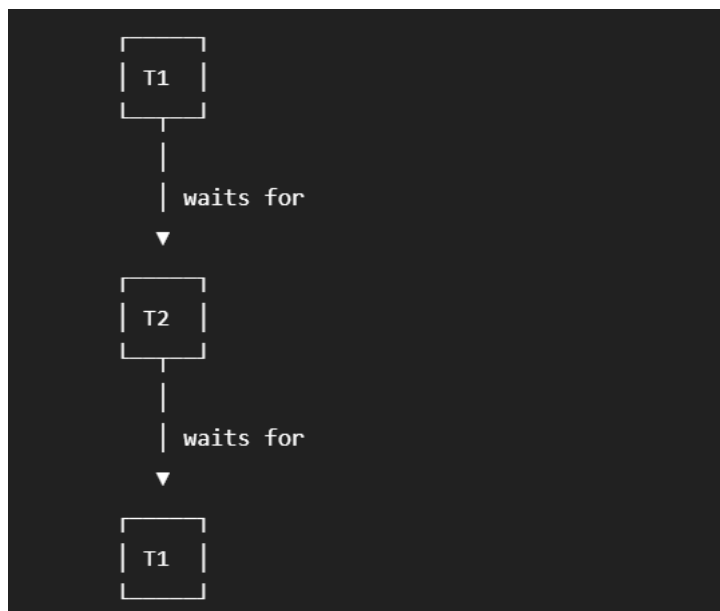
5. Deadlock Detection

The deadlock can be represented using a **wait-for graph**:



The cycle indicates the presence of a deadlock.

Figure 2: Wait-for Graph Showing Deadlock Between T1 and T2



6. Deadlock Resolution

One transaction can be selected as the **victim** and rolled back.

For example:

1. T1 holds lock on A and waits for B.
2. T2 holds lock on B and waits for A.
3. Deadlock is detected.

4. T2 is selected as the victim.
5. T2 is rolled back and its locks are released.
6. T1 obtains the required lock on B.
7. T1 completes and commits.
8. T2 is restarted later.

T1 → continues → COMMIT

T2 → ROLLBACK → RELEASE LOCKS → RESTART

7. Important Properties of 2PL

- A transaction must acquire locks during the **growing phase**.
- Once a lock is released, no new lock can be acquired.
- It ensures **conflict serializability**.
- Basic 2PL does **not completely prevent deadlocks**.
- Deadlocks can be detected using a wait-for graph and resolved by rollback.

8. Result

Basic Two-Phase Locking was applied to the given transactions. A deadlock was identified when T1 and T2 waited for each other's locks. The deadlock was resolved by rolling back one transaction and releasing its locks.

Conclusion

Basic 2PL ensures conflict-serializable schedules by controlling the acquisition and release of locks. However, it can result in deadlocks when transactions wait for each other's resources. Such deadlocks can be detected using a wait-for graph and resolved through transaction rollback.

QUESTION 6 – WAIT-DIE AND WOUND-WAIT DEADLOCK PREVENTION

Aim

To apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes to a given set of transactions requesting locks.

1. Given Transactions

Consider three transactions with the following timestamps:

Transaction	Timestamp	Age
T1	10	Older
T2	20	Younger
T3	30	Youngest

The lock requests are:

- T1 holds lock on A
- T2 requests lock on A
- T2 holds lock on B
- T3 requests lock on B

A smaller timestamp indicates an older transaction.

2. Wait-Die Scheme

Wait-Die is a non-preemptive deadlock prevention technique based on transaction timestamps.

Rules

- If an older transaction requests a lock held by a younger transaction, the older transaction waits.
- If a younger transaction requests a lock held by an older transaction, the younger transaction rolls back (dies).

Applying the Rule

Case 1: T2 requests A

T1 holds A and has timestamp 10.

T2 has timestamp 20.

Since T2 is younger than T1:

T2 → ROLLBACK (DIE)

Case 2: T3 requests B

T2 holds B and has timestamp 20.

T3 has timestamp 30.

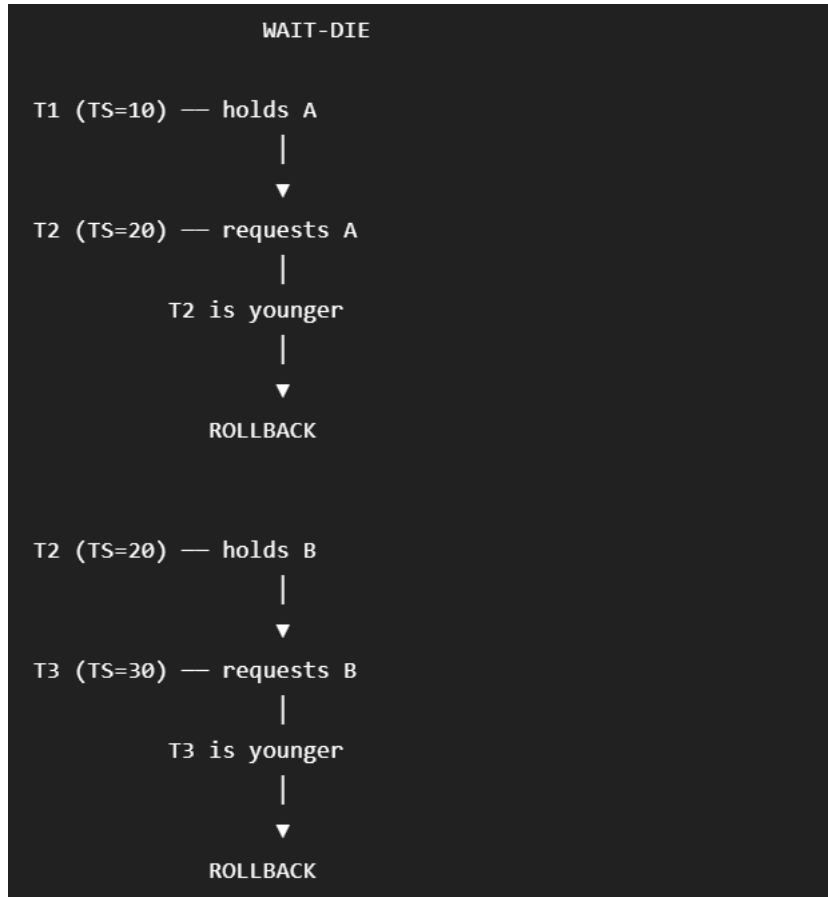
Since T3 is younger than T2:

T3 → ROLLBACK (DIE)

Wait-Die Result

Request	Lock Holder	Comparison	Action
T2 requests A	T1	T2 is younger	T2 rolls back
T3 requests B	T2	T3 is younger	T3 rolls back

Figure 1: Wait-Die Analysis



3. Wound-Wait Scheme

Wound-Wait is a **preemptive** deadlock prevention technique based on transaction timestamps.

Rules

- If an older transaction requests a lock held by a younger transaction, the older transaction wounds the younger transaction, causing it to roll back.
- If a younger transaction requests a lock held by an older transaction, the younger transaction waits.

Applying the Rule

Case 1: T2 requests A

T1 holds A and is older than T2.

T2 is younger than T1, so:

T2 → WAIT

Case 2: T3 requests B

T2 holds B and is older than T3.

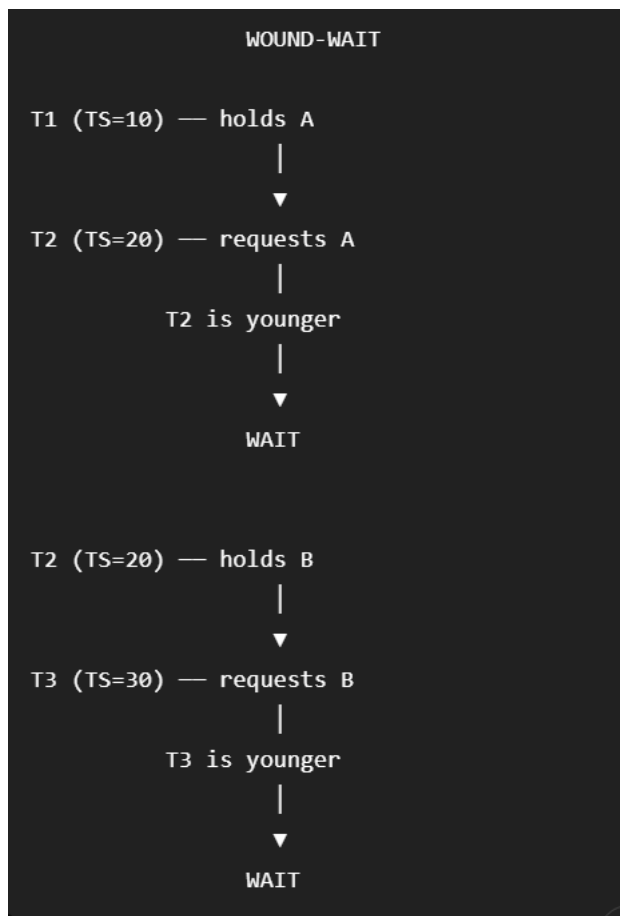
T3 is younger than T2, so:

T3 → WAIT

Wound-Wait Result

Request	Lock Holder	Comparison	Action
T2 requests A	T1	T2 is younger	T2 waits
T3 requests B	T2	T3 is younger	T3 waits

Figure 2: Wound-Wait Analysis



4. Comparison of Wait-Die and Wound-Wait

Feature	Wait-Die	Wound-Wait
Type	Non-preemptive	Preemptive
Older requests younger's lock	Wait	Younger rolls back
Younger requests older's lock	Rolls back	Waits
Uses timestamps	Yes	Yes
Prevents deadlock	Yes	Yes

5. Analysis

Both techniques use transaction timestamps to prevent circular waiting.

In Wait-Die, younger transactions are more likely to be rolled back when they request locks held by older transactions.

In Wound-Wait, older transactions have priority and can force younger transactions to roll back when necessary.

Thus, both techniques prevent deadlocks by maintaining an ordering based on transaction age.

6. Result

The Wait-Die and Wound-Wait schemes were successfully applied to the given transaction and lock requests. The appropriate WAIT or ROLLBACK action was determined using transaction timestamps.

Conclusion

Wait-Die and Wound-Wait are timestamp-based deadlock prevention techniques. Wait-Die is non-preemptive, whereas Wound-Wait is preemptive. Both techniques prevent deadlocks by controlling transaction waiting and rollback based on transaction age.

QUESTION 7 – IMMEDIATE UPDATE RECOVERY WITH CHECKPOINT

Aim

To apply the Immediate Update Recovery Protocol to recover the database after a system failure using transaction logs and checkpoints.

1. Given Transaction Log

Consider the following transaction log:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<START T3>

<T3, C, 300, 350>

<CHECKPOINT {T2, T3}>

<T2, B, 250, 280>

<T3, C, 350, 400>

<COMMIT T3>

--- SYSTEM FAILURE ---

Here:

- T1 committed before the checkpoint.
- T2 was active at the checkpoint and did not commit.
- T3 committed after the checkpoint.
- The system failure occurred after these operations.

2. Immediate Update Recovery

In the Immediate Update technique, database changes may be written to disk before the transaction commits.

Therefore, after a system failure:

- Committed transactions are REDONE if necessary.
- Uncommitted transactions are UNDONE to remove their effects.

The recovery process uses the transaction log to determine which transactions were committed and which were active at the time of failure.

3. Identify Transactions

From the given log:

Transaction	Status at Failure	Recovery Action
T1	Committed	REDO
T2	Uncommitted	UNDO
T3	Committed	REDO

4. Recovery Analysis

Transaction T1

T1 committed before the checkpoint.

<T1, A, 100, 150>

<COMMIT T1>

Since T1 is committed, its update is retained.

Action: REDO T1

Therefore:

A = 150

Transaction T2

T2 did not commit before the system failure.

<T2, B, 200, 250>

<T2, B, 250, 280>

Since T2 is uncommitted, its changes must be undone.

Undo operations are performed in reverse order:

B: 280 → 250

B: 250 → 200

Therefore:

B = 200

Action: UNDO T2

Transaction T3

T3 committed before the system failure.

<T3, C, 300, 350>

<T3, C, 350, 400>

<COMMIT T3>

Since T3 committed, its changes must be retained.

Action: REDO T3

Therefore:

C = 400

5. Recovery Process

The recovery process can be represented as:

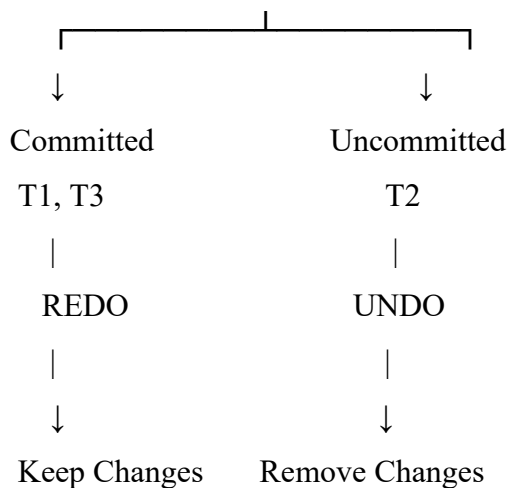
SYSTEM FAILURE

|

↓

Check Transaction Log

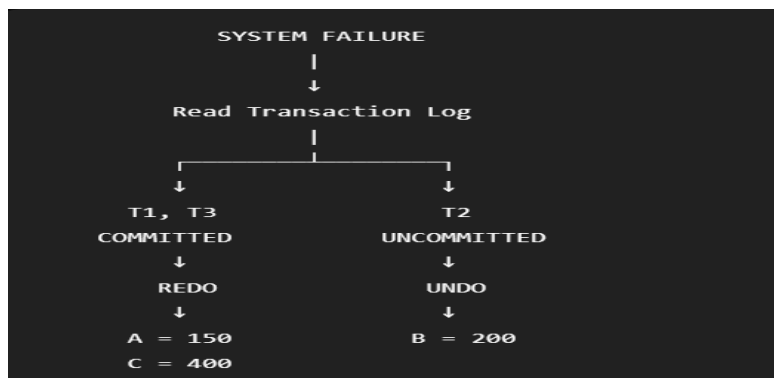
|



6. Final Database State

Data Item	Before Recovery	Final Value	Action
A	150	150	REDO T1
B	280	200	UNDO T2
C	400	400	REDO T3

Figure 1: Immediate Update Recovery Process After System Failure



7. Role of Checkpoint

A checkpoint reduces the amount of log that needs to be examined during recovery.

The checkpoint in the given log is:

<CHECKPOINT {T2, T3}>

At the checkpoint:

- T2 was still active.
- T3 was still active.
- T1 had already committed.

Therefore, the recovery manager can use the checkpoint to identify the transactions that may require recovery.

8. Recovery Algorithm

The basic recovery steps are:

1. Locate the most recent checkpoint in the log.
2. Identify committed and uncommitted transactions.
3. REDO the updates of committed transactions when required.
4. UNDO the updates of uncommitted transactions in reverse order.
5. Restore the database to a consistent state.

9. Result

After applying Immediate Update recovery:

A = 150

B = 200

C = 400

T1 and T3 are preserved because they committed, while T2 is rolled back because it was uncommitted at the time of failure.

Conclusion

Immediate Update recovery uses transaction logs and checkpoints to restore the database after a system failure. Committed transactions are redone when necessary, while uncommitted transactions are undone. In this example, T1 and T3 are retained and T2 is rolled back, resulting in a consistent database state.

QUESTION 8 – DEFERRED UPDATE (NO-UNDO/REDO) RECOVERY

Aim

To apply the Deferred Update (NO-UNDO/REDO) recovery technique to a transaction log containing committed and uncommitted transactions.

1. Given Transaction Log

Consider the following transaction log:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<START T3>

<T3, C, 300, 350>

<COMMIT T3>

<T2, B, 250, 280>

--- SYSTEM FAILURE ---

2. Deferred Update Recovery

In the Deferred Update technique, changes made by a transaction are not written to the database until the transaction commits.

Therefore:

- Committed transactions → REDO
- Uncommitted transactions → IGNORE
- UNDO is not required

This is why the technique is called NO-UNDO/REDO.

3. Identify Transaction Status

Transaction	Status at Failure	Recovery Action
T1	Committed	REDO
T2	Uncommitted	IGNORE
T3	Committed	REDO

4. Recovery Analysis

Transaction T1

T1 committed successfully:

<T1, A, 100, 150>

<COMMIT T1>

Therefore, T1's update is applied during recovery.

REDO T1

Final value:

A = 150

Transaction T2

T2 did not commit before the system failure.

<T2, B, 200, 250>

<T2, B, 250, 280>

Since the updates of an uncommitted transaction are not written to the database in deferred update:

T2 is ignored.

Therefore:

B = 200

No UNDO operation is required.

Transaction T3

T3 committed successfully:

<T3, C, 300, 350>

<COMMIT T3>

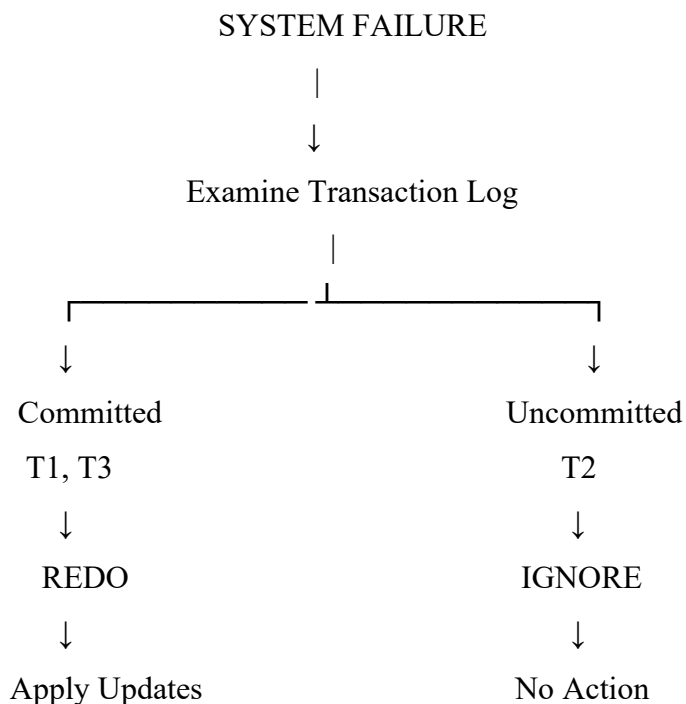
Therefore:

REDO T3

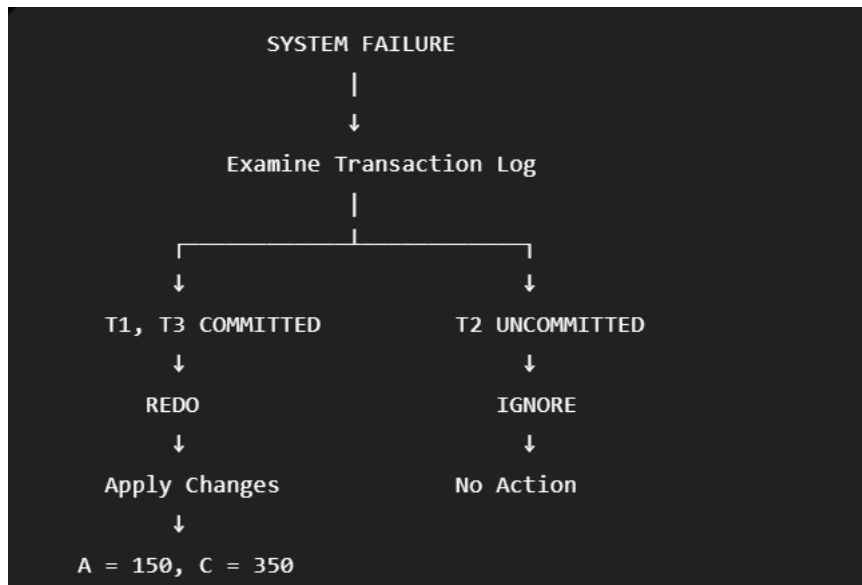
Final value:

C = 350

5. Recovery Process



Caption: Figure 1: Deferred Update Recovery Process



6. Final Database State

Data Item	Initial Value	Final Value	Recovery Action
A	100	150	REDO T1
B	200	200	IGNORE T2
C	300	350	REDO T3

7. Why UNDO Is Not Required

In Deferred Update recovery, database changes are postponed until the transaction commits. Therefore, an uncommitted transaction such as **T2** has not modified the database permanently. Hence, there is no need to undo T2's changes. Only committed transactions need to be REDONE.

8. Recovery Steps

1. Scan the transaction log.
2. Identify transactions that committed before the failure.
3. REDO the updates of committed transactions.
4. Ignore transactions that were active or uncommitted.
5. Restore the database to a consistent state.

9. Result

After applying Deferred Update recovery:

$A = 150$

$B = 200$

$C = 350$

T1 and T3 are recovered because they committed successfully. T2 is ignored because it was uncommitted at the time of system failure.

Conclusion

Deferred Update uses the NO-UNDO/REDO recovery technique. Since uncommitted updates are not written to the database, there is no need for UNDO operations. Only committed transactions are redone during recovery, allowing the database to be restored to a consistent state.

QUESTION 9 – CONCURRENCY CONTROL USING STRICT 2PL

Aim

To design a concurrency control mechanism using Strict Two-Phase Locking (Strict 2PL) for a banking system where one transaction transfers funds and another transaction reads the balance simultaneously.

1. Banking Scenario

Consider a bank account A with an initial balance of ₹10,000.

Two transactions are executed concurrently:

- **T1:** Transfers ₹2,000 from Account A to Account B.
- **T2:** Reads the balance of Account A.

The objective is to ensure that T2 does not read an inconsistent or partially updated balance while T1 is transferring the money.

2. Transactions

Transaction T1 – Fund Transfer

T1:

LOCK-X(A)

READ(A)

$A = A - 2000$

WRITE(A)

LOCK-X(B)

READ(B)

$B = B + 2000$

WRITE(B)

COMMIT

UNLOCK(A)

UNLOCK(B)

Transaction T2 – Balance Reading

T2:

LOCK-S(A)

READ(A)

UNLOCK(A)

COMMIT

3. Strict 2PL

In Strict Two-Phase Locking:

- A transaction can acquire locks during the growing phase.
- Exclusive locks are held until the transaction commits or aborts.
- Shared locks may also be held until commit in strict implementations.
- This prevents other transactions from reading uncommitted data.

4. Execution of T1 and T2

Initial balance:

A = ₹10,000

T1 starts the transfer:

T1 → LOCK-X(A)

T1 → READ(A)

T1 → A = ₹8,000

T1 → WRITE(A)

At this point, T2 requests a shared lock on A:

T2 → LOCK-S(A)

T2 cannot obtain the lock because T1 currently holds an exclusive lock.

Therefore:

T2 → WAIT

T1 continues:

T1 → LOCK-X(B)

T1 → READ(B)

T1 → B = B + ₹2,000

T1 → WRITE(B)

T1 → COMMIT

T1 → UNLOCK(A)

T1 → UNLOCK(B)

After T1 releases the lock, T2 can continue:

T2 → LOCK-S(A)

T2 → READ(A)

T2 → UNLOCK(A)

T2 → COMMIT

Therefore, T2 reads the final committed balance of: ₹8,000

6. Strict 2PL Schedule

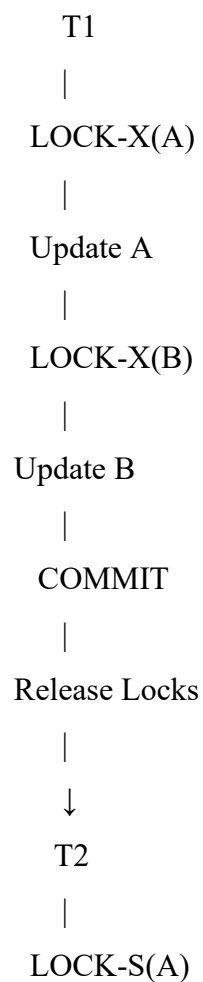
Step	T1 – Transfer	T2 – Read Balance
1	LOCK-X(A)	
2	READ(A)	
3	WRITE(A = ₹8,000)	
4	LOCK-X(B)	
5	WRITE(B)	
6	COMMIT	
7	UNLOCK(A), UNLOCK(B)	
8		LOCK-S(A)
9		READ(A) = ₹8,000
10		UNLOCK(A)
11		COMMIT

Figure 1: Strict 2PL Schedule for Fund Transfer and Balance Reading



6. Locking Behavior

The locking mechanism can be represented as:



|
READ(A)
|
COMMIT

T2 waits until T1 commits because T1's exclusive lock prevents T2 from reading the account while the transfer is incomplete.

7. Why Strict 2PL Is Required

Strict 2PL prevents dirty reads and ensures that transactions do not read uncommitted data. Without strict locking, T2 might read the temporary balance of ₹8,000 before T1 finishes updating Account B. If T1 later fails, T2 would have read an incorrect value. Strict 2PL prevents this situation by making T2 wait until T1 commits.

8. Advantages

- Prevents dirty reads.
- Maintains database consistency.
- Ensures conflict serializability.
- Prevents cascading rollback caused by reading uncommitted data.
- Suitable for financial transactions where consistency is critical.

9. Result

Strict 2PL was applied to the banking scenario. T1 obtains an exclusive lock during the fund transfer, while T2 waits before reading the balance. After T1 commits and releases its locks, T2 reads the updated balance of ₹8,000.

Conclusion

Strict Two-Phase Locking provides safe concurrency control for banking transactions. By holding exclusive locks until commit, it prevents T2 from reading intermediate or uncommitted values. Thus, the banking system maintains consistency, isolation, and reliable transaction execution.

QUESTION 10 – PSEUDOCODE FOR HEURISTIC QUERY OPTIMIZATION

Aim

To design pseudocode for query optimization using heuristic rules such as selection pushdown, early projection, and ordering joins to produce smaller intermediate results.

1. Heuristic Query Optimization

Heuristic query optimization improves query performance by applying predefined rules to the query execution plan instead of calculating the exact cost of every possible plan.

The main rules used are:

1. Push selections down as close as possible to the base relations.
2. Perform projections early to remove unnecessary attributes.
3. Order joins so that smaller intermediate relations are produced first.

ALGORITHM :Heuristic_Query_Optimization

INPUT:

Query Q containing selections, projections and joins

OUTPUT:

Optimized query execution plan

BEGIN

Step 1: Convert Q into a relational algebra expression and construct the initial query tree.

Step 2: Push selection operations downward.

Move each selection as close as possible to the corresponding base relation.

Step 3: Push projection operations downward.

Keep only attributes required for:

- selection conditions
- join conditions
- final output

Step 4: Identify all join operations.

Step 5: Estimate the size of intermediate relations.

Step 6: Order the joins.

Perform joins involving smaller relations first.

Step 7: Reconstruct the optimized query tree.

Step 8: Return the optimized execution plan.

END

3. Selection Pushdown

Selection operations are performed as early as possible.

Example

Before:

```
      JOIN
     /   \
Student  Department
   |
Selection
City='Chennai'
```

After:

```
      JOIN
     /   \
Selection  Department
   |
Student
```

Benefit: Unwanted tuples are removed before the join, reducing intermediate results.

4. Projection Pushdown

Projection is moved closer to the base tables.

Only attributes required for joins, selections, and final output are retained.

Student → RegNo, Name, DeptID

Department → DeptID, DeptName

Benefit: It reduces the number of attributes and the amount of data processed.

5. Join Ordering

Joins should be performed in an order that produces smaller intermediate relations.

Relation A

+

Small Relation B

↓

Small Intermediate Result

+

Relation C

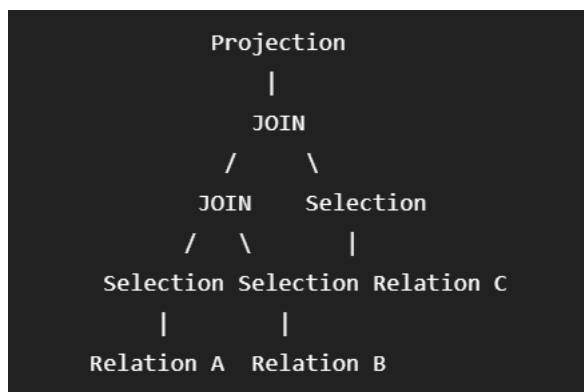
↓

Final Result

Benefit: Smaller intermediate results reduce memory usage and disk I/O.

6. Optimized Query Tree

The general optimized structure is:



This structure ensures that selections and projections are applied before expensive join operations.

7. Advantages of Heuristic Optimization

- Reduces the number of tuples processed.
- Reduces the size of intermediate relations.
- Reduces memory requirements.
- Reduces disk I/O.
- Improves query execution performance.
- Produces a simpler and more efficient execution plan.

8. Result

A pseudocode algorithm for heuristic query optimization was designed using selection pushdown, projection pushdown, and join ordering. These techniques reduce unnecessary processing and intermediate results.

Conclusion

Heuristic query optimization improves query performance by applying simple but effective transformation rules. Selection is pushed down, required attributes are projected early, and joins are ordered to produce smaller intermediate results. These techniques help the DBMS execute queries more efficiently.